UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

Sede Panamá Oeste

Facultad de Ingeniería en Sistemas Computacionales

Lic. en Desarrollo y Gestión de Software

INGENIERÍA DE SOFTWARE APLICADA I

TRABAJO DE INVESTIGACIÓN

Tema:

LEVANTAMIENTO DE REQUERIMIENTOS

Profesora:

Susan Janette Oliva Rivera

Integrantes:

Luis Madrid	Cédula: 8-995-2449
José Sousa	Cédula: 8-823-836
Roderick Justiniani	Cédula: 8-807-791
Luis Montalvo	Cédula: 8-1012-864

Panamá Oeste, República de Panamá

ÍNDICE

INTRODUCCIÓN i

OBJETIVOS iii

Objetivo general iii

Objetivos específicos iii

JUSTIFICACIÓN (RELEVANCIA DEL ESTUDIO) iv

CONTENIDO v

1. Fundamentos del levantamiento de requerimientos v

1.1 Concepto de requerimiento v

1.2 Concepto de levantamiento de requerimientos v

1.3 El levantamiento dentro de la ingeniería de requerimientos vi

1.4 Importancia y consecuencias de un levantamiento deficiente vi

2. Tipos de requerimientos vii

2.1 Requerimientos de negocio, de usuario y de sistema vii

2.2 Requerimientos funcionales viii

2.3 Requerimientos no funcionales ix

2.4 Reglas de negocio, requerimientos de dominio e interfaces externas x

3. Actores involucrados en el levantamiento x

3.1 Identificación de los actores (stakeholders) x

3.2 El rol del analista de requerimientos xi

3.3 Análisis de los actores y fuentes de requerimientos xii

4. El proceso de levantamiento de requerimientos xiii

4.1 Identificación de la necesidad y del alcance xiv

4.2 Obtención de los requerimientos xiv

4.3 Análisis y elaboración xiv

4.4 Negociación y priorización xv
4.5 Especificación xv
4.6 Validación xv
4.7 Gestión de cambios y trazabilidad xvi
5. Técnicas de recopilación de información xvi
5.1 Entrevistas xvii
5.2 Cuestionarios y encuestas xvii
5.3 Observación y etnografía xviii
5.4 Talleres de trabajo (JAD) y lluvia de ideas xviii
5.5 Análisis de documentos y de sistemas existentes xix
5.6 Prototipos xix
5.7 Escenarios, casos de uso e historias de usuario xix
5.8 Criterios para seleccionar las técnicas xx
6. Documentación y especificación de requerimientos xxi
6.1 El documento de especificación de requerimientos de software (ERS) xxi
6.2 Características de un buen requerimiento xxi
6.3 Formas de redactar los requerimientos xxii
6.4 Ejemplo de especificación xxiv
7. Priorización y validación de requerimientos xxiv
7.1 ¿Por qué priorizar? xxiv
7.2 La técnica MoSCoW xxv
7.3 Validación de los requerimientos xxv
8. El levantamiento en los enfoques tradicionales y ágiles xxvi
8.1 Enfoque tradicional xxvi
8.2 Enfoque ágil xxvi

8.3 Comparación xxvii
9. Problemas frecuentes, buenas prácticas y herramientas xxviii
9.1 Problemas frecuentes xxviii
9.2 Buenas prácticas xxix
9.3 Herramientas de apoyo xxix
9.4 Consideraciones éticas y de inclusión xxx
CONCLUSIONES Y RECOMENDACIONES xxxi
Luis Madrid xxxi
Conclusiones xxxi
El levantamiento de requerimientos es la etapa que define el rumbo de un proyecto: de su calidad dependen el diseño, las pruebas y la satisfacción de los usuarios. xxxi
Un requerimiento es una necesidad comprendida y expresada con claridad; por eso, levantar requerimientos implica descubrir, interpretar y acordar, y no solo recolectar solicitudes. xxxi
Recomendaciones xxxi
Dedicar tiempo suficiente al levantamiento antes de iniciar el diseño, definiendo desde el principio el alcance, los actores y el calendario de reuniones. xxxi
Detectar los errores de requerimientos lo más temprano posible, ya que corregirlos tarde es mucho más costoso. xxxi
José Sousa xxxi
Conclusiones xxxi
Distinguir entre requerimientos de negocio, de usuario y de sistema, y entre funcionales y no funcionales, permite organizar la información y evitar que aspectos como el rendimiento o la seguridad se descubran demasiado tarde. xxxi

Identificar a todos los actores y comprender su nivel de interés e influencia es indispensable para obtener información completa y anticipar conflictos que deberán negociarse. xxxi

Recomendaciones xxxi

Analizar los requerimientos no funcionales desde el inicio del proyecto y no dejarlos para el final. xxxi

Elaborar al comenzar una lista de actores y una matriz de interés e influencia, para saber a quién consultar y con qué frecuencia. xxxii

Roderick Justiniani xxxii

Conclusiones xxxii

El proceso de levantamiento es iterativo: sus etapas se repiten y se solapan, y volver a etapas anteriores cuando surgen cambios es normal, no un fallo. xxxii

No existe una técnica que sirva para todos los casos; combinar entrevistas, cuestionarios, observación, talleres, análisis de documentos y prototipos produce resultados más confiables. xxxii

Recomendaciones xxxii

Elegir las técnicas según el contexto del proyecto (número de usuarios, tiempo, presupuesto y novedad del sistema) y combinar más de una. xxxii

Preparar guías de preguntas y enviar a los participantes un resumen de cada reunión para corregir malentendidos a tiempo. xxxii

Luis Montalvo xxxii

Conclusiones xxxii

Documentar los requerimientos con criterios de calidad (claros, verificables, singulares, priorizados y trazables) reduce las ambigüedades y facilita la validación con los usuarios. xxxii

Tanto los enfoques tradicionales como los ágiles necesitan levantar requerimientos; cambian el momento y el nivel de formalidad, pero en ambos es indispensable gestionar los cambios de forma ordenada. xxxii

Recomendaciones xxxiii

Redactar cada requerimiento con un identificador único, una sola idea y criterios medibles, evitando términos vagos como “rápido” o “fácil”, y priorizarlos con técnicas como MoSCoW. xxxiii

Establecer desde el inicio una línea base, un procedimiento de control de cambios y una matriz de trazabilidad, incluso en proyectos pequeños. xxxiii

INFOGRAFÍA xxxiv

BIBLIOGRAFÍA xxxv

ANEXOS xxxviii

Anexo A. Guía modelo para una entrevista de levantamiento xxxviii

Anexo B. Plantilla de ficha de requerimiento xxxix

Anexo C. Glosario de términos xl

INTRODUCCIÓN

Luis Madrid

Todo proyecto de software nace de un problema que alguien necesita resolver, y entenderlo bien es el primer paso para lograrlo. El levantamiento de requerimientos es la etapa en la que se descubre qué necesitan realmente los usuarios antes de diseñar o programar. Su importancia es tal que, según el Project Management Institute (2014), 47 % de los proyectos no exitosos no alcanzó sus metas por una gestión inadecuada de los requerimientos. Mi parte de la investigación presenta los conceptos básicos del levantamiento de requerimientos y explica por qué es tan importante para el éxito de un proyecto.

José Sousa

Para que un sistema responda a lo que se espera de él, es necesario saber con claridad qué debe hacer y cómo debe comportarse. En mi parte de la investigación explico los tipos de requerimientos (de negocio, de usuario, de sistema, funcionales y no funcionales) y presento a los actores involucrados, como los usuarios, los clientes y el analista de requerimientos, cuyas necesidades y puntos de vista determinan lo que finalmente se construye.

Roderick Justiniani

Levantar requerimientos no es una simple conversación con el cliente: exige un proceso ordenado y técnicas adecuadas. En mi parte de la investigación describo las etapas del proceso (identificar, obtener, analizar, negociar, especificar y validar) y las principales técnicas de recopilación de información, como las entrevistas, los cuestionarios, la observación, los talleres y los prototipos, junto con los criterios para elegir la más apropiada.

Luis Montalvo

Una vez obtenidos, los requerimientos deben documentarse, priorizarse y validarse para que todos los involucrados los entiendan de la misma manera. En mi parte de la investigación explico cómo se redactan y especifican, cómo se priorizan con la técnica MoSCoW, cómo se validan y cómo cambia el proceso en los enfoques

tradicionales y ágiles. Termino con los problemas más frecuentes y las buenas prácticas.

OBJETIVOS

Objetivo general

Analizar el proceso de levantamiento de requerimientos de software, sus técnicas y sus buenas prácticas, con el fin de comprender su importancia en el desarrollo de sistemas de calidad que respondan a las necesidades de los usuarios.

Objetivos específicos

- Definir los conceptos de requerimiento, levantamiento de requerimientos e ingeniería de requerimientos.
- Diferenciar los tipos de requerimientos: de negocio, de usuario, de sistema, funcionales y no funcionales.
- Identificar a los actores involucrados en un proyecto y el papel del analista de requerimientos.
- Describir las etapas del proceso de levantamiento, desde la identificación de la necesidad hasta la validación.
- Comparar las principales técnicas de recopilación de información y los criterios para seleccionarlas.
- Explicar cómo se especifican, priorizan y validan los requerimientos, tanto en enfoques tradicionales como ágiles.
- Reconocer los problemas más frecuentes del levantamiento y las buenas prácticas para evitarlos.

JUSTIFICACIÓN (RELEVANCIA DEL ESTUDIO)

Un sistema de software se considera exitoso cuando resuelve el problema para el cual fue creado, dentro del tiempo y del presupuesto acordados. Ninguno de esos tres elementos puede lograrse si el equipo no tiene claro qué debe construir. El levantamiento de requerimientos es la etapa que proporciona esa claridad y, por ello, constituye la base sobre la cual se apoyan el diseño, la programación, las pruebas y el mantenimiento.

Además, la falta de requerimientos bien definidos tiene un costo económico medible. El Project Management Institute (2014) estimó que, por cada dólar invertido en proyectos y programas, 5,1 % se desperdicia por una mala gestión de los requerimientos; esto equivale a unos 51 millones de dólares por cada 1 000 millones invertidos. Aunque la cifra proviene de un estudio sobre gestión de proyectos en general y no solo de proyectos de software, ilustra la magnitud del problema y la conveniencia de dedicar tiempo y recursos a esta etapa.

Para el estudiante de ingeniería de software y carreras afines, dominar el levantamiento de requerimientos es una competencia profesional básica. Tanto el analista de sistemas como el desarrollador, el responsable de pruebas o el líder de proyecto interactúan con los requerimientos a lo largo de su trabajo. Conocer las técnicas de obtención, los criterios de calidad y las formas de documentación permite comunicarse mejor con clientes y compañeros, reducir el retrabajo y entregar productos que realmente satisfacen a los usuarios.

Por último, en un contexto en el que cada vez más empresas, instituciones educativas y entidades públicas digitalizan sus procesos, saber levantar requerimientos ayuda a que la tecnología responda a necesidades reales y no a suposiciones. Esta investigación busca, en consecuencia, ofrecer una visión ordenada y fundamentada del tema que pueda aplicarse tanto en los proyectos de la asignatura como en el ejercicio profesional futuro.

CONTENIDO

1. Fundamentos del levantamiento de requerimientos

1.1 Concepto de requerimiento

Un requerimiento es una declaración que expresa una necesidad de los usuarios o de la organización, junto con las condiciones y restricciones asociadas, y que el sistema debe satisfacer (ISO/IEC/IEEE, 2018). Sommerville (2011) explica que los requerimientos abarcan tanto los servicios que el sistema ofrece como las restricciones bajo las cuales opera, y que pueden expresarse desde una frase abstracta de alto nivel hasta una especificación funcional detallada.

Esa amplitud explica por qué un mismo término se utiliza en niveles muy distintos. Por ejemplo, “los estudiantes deben poder consultar sus calificaciones en línea” es una necesidad expresada en lenguaje cotidiano, mientras que “el sistema mostrará al estudiante autenticado la lista de calificaciones del período en curso” es una declaración más precisa que el equipo puede diseñar, construir y probar. Ambas son requerimientos, pero cumplen funciones diferentes dentro del proyecto.

1.2 Concepto de levantamiento de requerimientos

El levantamiento de requerimientos es el conjunto de actividades mediante las cuales el equipo de desarrollo descubre, comprende y registra lo que necesitan los actores involucrados en un proyecto de software. En la literatura en inglés se conoce como *requirements elicitation*, y en español se emplean también los términos obtención, captura, educción, recolección o toma de requerimientos. En América Latina el término “levantamiento” es el más habitual en el ámbito académico y profesional.

No se trata simplemente de preguntar al cliente qué desea. Los usuarios no siempre saben con claridad lo que necesitan, describen su trabajo con el vocabulario propio de su área, dan por obvias ciertas cosas que el analista desconoce o, sencillamente, no han imaginado todavía qué podría hacer un sistema por ellos. Por esa razón, Sommerville (2011) habla de obtención y análisis de requerimientos como un solo

proceso: no basta con recolectar información, es necesario interpretarla, contrastarla y organizarla.

1.3 El levantamiento dentro de la ingeniería de requerimientos

La ingeniería de requerimientos es la disciplina que abarca todas las actividades necesarias para descubrir, analizar, documentar y verificar los servicios que requiere el cliente y las restricciones bajo las cuales el sistema debe operar (Kotonya & Sommerville, 1998). El levantamiento es su primera actividad, pero no la única: el proceso continúa con el análisis, la especificación, la validación y la gestión de los requerimientos a lo largo de la vida del proyecto.

La Guía del Cuerpo de Conocimiento de la Ingeniería de Software (SWEBOK) dedica un área completa a los requerimientos de software y la organiza en fundamentos, proceso, obtención, análisis, especificación, validación, consideraciones prácticas y herramientas (Bourque & Fairley, 2014). Esto confirma que el levantamiento forma parte de un conjunto más amplio de actividades que se influyen entre sí y que suelen repetirse de manera iterativa.

1.4 Importancia y consecuencias de un levantamiento deficiente

Pressman (2010) y Sommerville (2011) coinciden en que el costo de corregir un error crece a medida que el proyecto avanza. Un requerimiento mal entendido que se detecta durante la entrevista se corrige con una conversación; si se detecta durante las pruebas, obliga a rediseñar, reprogramar y volver a probar; y si se detecta cuando el sistema ya está en uso, puede implicar interrupciones del servicio, pérdida de confianza y costos de mantenimiento elevados.

Entre las consecuencias más comunes de un levantamiento deficiente se encuentran las siguientes:

- **Retrabajo y sobrecostos:** se construyen funciones que luego deben modificarse o eliminarse.
- **Retrasos en la entrega:** los cambios tardíos alteran la planificación y obligan a reasignar recursos.

- **Funciones innecesarias:** el sistema incluye opciones que nadie utiliza, mientras que faltan otras que sí se necesitan.
- **Insatisfacción de los usuarios:** el producto cumple con lo escrito, pero no con lo que las personas esperaban.
- **Conflictos contractuales:** si no existe una definición clara del alcance, cliente y proveedor interpretan los compromisos de forma distinta.

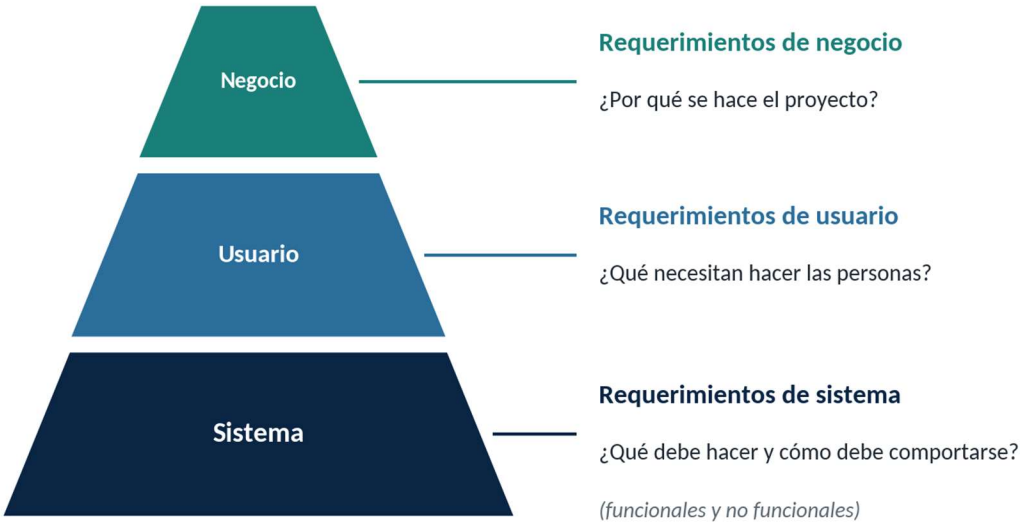
Por el contrario, cuando el levantamiento se realiza con rigor se logra un entendimiento compartido entre todos los involucrados, se cuenta con una base sólida para estimar el esfuerzo y planificar el trabajo, se facilita el diseño de las pruebas de aceptación y se reduce el riesgo de que el proyecto se abandone antes de concluir.

2. Tipos de requerimientos

2.1 Requerimientos de negocio, de usuario y de sistema

Los requerimientos pueden clasificarse por el nivel de detalle y por la audiencia a la que se dirigen. Wiegers y Beatty (2013) distinguen los requerimientos de negocio, que explican por qué se emprende el proyecto y qué beneficio se espera para la organización; los requerimientos de usuario, que describen lo que las personas necesitan lograr con el sistema; y los requerimientos funcionales, que detallan el comportamiento que el software debe tener. De forma similar, Sommerville (2011) diferencia los requerimientos de usuario, redactados en lenguaje natural y con diagramas para que los comprendan clientes y usuarios, y los requerimientos de sistema, más detallados y dirigidos a quienes diseñan y construyen la solución.

Figura 1
Niveles de los requerimientos



Un ejemplo permite ver la relación entre niveles. En un proyecto para una biblioteca universitaria, el requerimiento de negocio podría ser “reducir el tiempo que el personal dedica a controlar los préstamos”; el requerimiento de usuario, “el bibliotecario necesita registrar un préstamo en pocos pasos”; y el requerimiento de sistema, “el sistema permitirá registrar un préstamo seleccionando al estudiante y el libro, y calculará la fecha de devolución automáticamente”. Cada nivel se deriva del anterior y lo hace más concreto.

2.2 Requerimientos funcionales

Los requerimientos funcionales son declaraciones de los servicios que el sistema debe proporcionar, de la manera en que debe reaccionar ante entradas particulares y de cómo debe comportarse en situaciones específicas (Sommerville, 2011). Responden a la pregunta “¿qué debe hacer el sistema?”. Se refieren a funciones, procesos, cálculos, reglas de validación, reportes, consultas y manejo de datos.

Algunos ejemplos son: “el sistema permitirá registrar un nuevo estudiante con su cédula, nombre y correo electrónico”, “el sistema generará un reporte mensual de los libros más prestados” y “el sistema enviará un mensaje de confirmación cuando

se complete una reserva". Cada uno describe una capacidad concreta que puede verificarse observando el funcionamiento del sistema.

2.3 Requerimientos no funcionales

Los requerimientos no funcionales no describen servicios específicos, sino las propiedades y restricciones bajo las cuales el sistema debe operar: su rendimiento, su seguridad, su facilidad de uso, su disponibilidad, entre otras (Sommerville, 2011). Responden a la pregunta "¿cómo debe comportarse el sistema?" y con frecuencia determinan el éxito del producto tanto como las funciones mismas: un sistema con todas las funciones correctas, pero lento o inseguro, será rechazado por sus usuarios.

Sommerville (2011) agrupa estos requerimientos en tres categorías: requerimientos del producto (rendimiento, fiabilidad, seguridad, usabilidad), requerimientos organizacionales (estándares, procesos de desarrollo, entorno de operación) y requerimientos externos (legales, éticos, de interoperabilidad con otros sistemas). La Tabla 1 resume las categorías más habituales con ejemplos.

Tabla 1
Categorías frecuentes de requerimientos no funcionales

Categoría	Qué describe	Ejemplo
Rendimiento	Tiempos de respuesta, capacidad y volumen de usuarios	Las búsquedas devolverán resultados en menos de 3 segundos con 200 usuarios simultáneos.
Usabilidad	Facilidad de aprendizaje y de uso	Un usuario nuevo podrá registrar un préstamo sin haber recibido capacitación previa.
Seguridad	Autenticación, autorización y protección de datos	El acceso al módulo de administración requerirá usuario y contraseña de al menos 8 caracteres.
Disponibilidad y fiabilidad	Continuidad del servicio y tolerancia a fallos	El sistema estará disponible al menos 99 % del horario de atención.
Mantenibilidad	Facilidad para corregir o ampliar el sistema	Cada módulo contará con documentación técnica actualizada.
Compatibilidad y portabilidad	Entornos y dispositivos en los que funciona	La aplicación funcionará en las dos últimas versiones de los navegadores más usados.

Categoría	Qué describe	Ejemplo
Legales y normativos	Leyes, normas y políticas que deben cumplirse	Los datos personales se tratarán conforme a la normativa vigente.

Nota. Elaboración propia con base en Sommerville (2011) y Wiegers y Beatty (2013).

Es importante señalar que la frontera entre ambos tipos no siempre es nítida. Un requerimiento de seguridad, como la protección de la información de los estudiantes, puede dar origen a varios requerimientos funcionales: inicio de sesión, asignación de roles y registro de accesos. Por esa razón conviene analizar los requerimientos no funcionales desde el inicio y no dejarlos para el final del proyecto, cuando modificar la arquitectura del sistema resulta muy costoso.

2.4 Reglas de negocio, requerimientos de dominio e interfaces externas

Además de las categorías anteriores, Wiegers y Beatty (2013) llaman la atención sobre otros elementos que el levantamiento debe capturar. Las **reglas de negocio** son políticas, normas o cálculos propios de la organización que el sistema debe respetar; por ejemplo, “un estudiante no puede tener más de tres libros prestados al mismo tiempo”. Los **requerimientos del dominio** provienen del área de aplicación (educación, salud, banca, logística) y reflejan sus prácticas y su terminología. Las **interfaces externas** describen cómo el sistema se comunica con otros sistemas, dispositivos o usuarios, y las **restricciones** son limitaciones impuestas al diseño, como el uso de determinada tecnología, un presupuesto máximo o una fecha de entrega.

3. Actores involucrados en el levantamiento

3.1 Identificación de los actores (stakeholders)

Un actor o *stakeholder* es cualquier persona, grupo u organización que se ve afectado por el sistema o que puede influir en él. Identificarlos correctamente es el primer paso del levantamiento, ya que una necesidad que no se expresa porque su portador no fue consultado se convierte, tarde o temprano, en un requerimiento omitido (Sommerville, 2011). Entre los actores más comunes se encuentran:

- **Usuarios finales:** quienes interactúan directamente con el sistema en su trabajo diario.
- **Clientes y patrocinadores:** quienes financian el proyecto y aprueban sus resultados.
- **Gerentes y directivos:** quienes definen los objetivos estratégicos y las prioridades de la organización.
- **Expertos del dominio:** especialistas que conocen las reglas y los procesos del área de aplicación.
- **Equipo de desarrollo y de pruebas:** quienes construyen, verifican y mantienen el sistema.
- **Personal de soporte y operaciones:** quienes instalarán, administrarán y darán asistencia a los usuarios.
- **Entes reguladores y auditores:** quienes exigen el cumplimiento de leyes, normas y estándares.

También deben considerarse los usuarios indirectos, es decir, quienes no utilizan el sistema, pero dependen de su información o de sus resultados, como los padres de familia que reciben el boletín de un estudiante o los proveedores que reciben órdenes de compra generadas automáticamente.

3.2 El rol del analista de requerimientos

El analista de requerimientos (también llamado analista de sistemas o analista de negocio) es el intermediario entre el mundo de los usuarios y el mundo técnico. Su tarea consiste en planificar el levantamiento, aplicar las técnicas de obtención, analizar la información recogida, redactar la especificación y verificar con los actores que lo escrito refleja sus necesidades (Kendall & Kendall, 2011; Wiegers & Beatty, 2013).

Para cumplir con esa responsabilidad se requieren tanto habilidades técnicas como personales. Entre ellas se destacan la escucha activa, la capacidad de formular buenas preguntas, el pensamiento analítico para detectar contradicciones y vacíos, la comunicación oral y escrita, la negociación, la empatía con quienes no dominan la tecnología, el conocimiento básico del dominio del problema y la discreción para

manejar información confidencial. Un analista que domina las herramientas, pero no sabe escuchar difícilmente obtendrá requerimientos de calidad.

3.3 Análisis de los actores y fuentes de requerimientos

Una vez identificados, los actores no tienen el mismo grado de interés ni de influencia sobre el proyecto. Una herramienta muy utilizada en la gestión de proyectos es la matriz de interés e influencia, que permite decidir cuánta atención dedicar a cada grupo y con qué frecuencia comunicarse con él. La Tabla 2 muestra una adaptación aplicada a un proyecto de software.

Tabla 2
Matriz de interés e influencia de los actores

Nivel de interés e influencia	Estrategia	Ejemplos
Alta influencia y alto interés	Gestionar de cerca	Patrocinador del proyecto, jefe del área usuaria, usuarios clave.
Alta influencia y bajo interés	Mantener satisfechos	Dirección general, entes reguladores, área de tecnología de la organización.
Baja influencia y alto interés	Mantener informados	Usuarios finales, personal de soporte, estudiantes o clientes que usarán el sistema.
Baja influencia y bajo interés	Monitorear	Áreas que se relacionan con el sistema de forma ocasional.

Nota. Elaboración propia, adaptada de la práctica habitual en la gestión de proyectos.

Los requerimientos no provienen únicamente de las personas. Otras fuentes importantes son los documentos de la organización (manuales, formularios, reportes, procedimientos), los sistemas existentes que se van a reemplazar o con los que se debe integrar, las leyes y normas aplicables, y los productos de la competencia o de organizaciones similares. Combinar varias fuentes permite contrastar la información y descubrir diferencias entre lo que se dice que se hace y lo que realmente se hace.

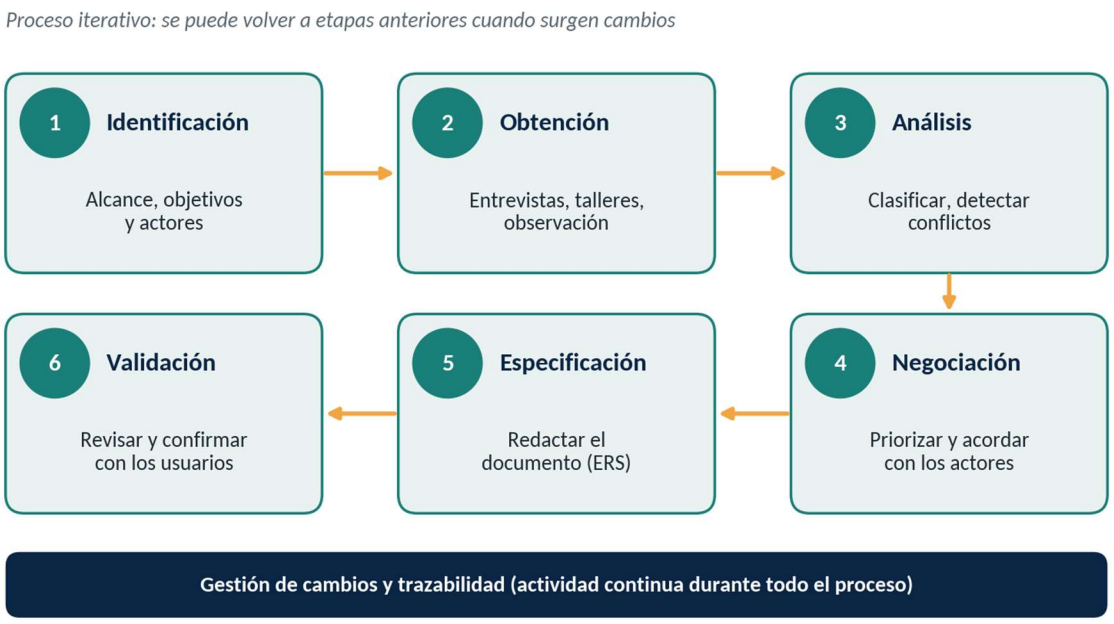
Por último, es frecuente que los actores tengan necesidades contradictorias: el área de ventas puede querer un proceso ágil y con pocos controles, mientras que el área de auditoría exige múltiples aprobaciones. Detectar estos conflictos y llevarlos a una

negociación explícita es una de las funciones centrales del analista, como se verá en el capítulo siguiente.

4. El proceso de levantamiento de requerimientos

No existe un único proceso válido para todos los proyectos. Sommerville (2011) organiza la ingeniería de requerimientos en cuatro actividades: estudio de factibilidad, obtención y análisis, especificación y validación. Pressman (2010), por su parte, propone siete tareas: concepción, indagación, elaboración, negociación, especificación, validación y administración de los requerimientos. Tomando como base ambos modelos, en este trabajo se presenta un proceso de seis etapas, acompañado de una actividad continua de gestión de cambios, que se resume en la Figura 2.

Figura 2
Etapas del proceso de levantamiento de requerimientos



Aunque las etapas se presentan en secuencia, en la práctica se solapan y se repiten. Al validar un requerimiento con los usuarios es común descubrir uno nuevo, lo que obliga a volver a la obtención y al análisis. Esta naturaleza iterativa es normal y no debe interpretarse como un fallo del proceso.

4.1 Identificación de la necesidad y del alcance

El proceso comienza con una comprensión general del problema y de su contexto. En esta etapa se responde a preguntas básicas: qué problema u oportunidad motiva el proyecto, quiénes son los actores, qué se espera lograr, qué queda dentro y fuera del alcance, y si el proyecto es factible desde el punto de vista técnico, económico y organizacional. Sommerville (2011) denomina a esta actividad estudio de factibilidad y señala que debe ser breve y barata, pues su propósito es decidir si vale la pena continuar.

Los productos típicos de esta etapa son un documento de visión y alcance, una lista preliminar de actores, un glosario inicial de términos y una primera identificación de riesgos. Definir el alcance desde el principio evita uno de los problemas más comunes de los proyectos: la ampliación descontrolada de funciones a medida que avanza el desarrollo.

4.2 Obtención de los requerimientos

La obtención es el núcleo del levantamiento: consiste en aplicar técnicas para recopilar información de los actores y de otras fuentes. Requiere una preparación previa: elegir las técnicas más adecuadas, elaborar guías de preguntas, programar las sesiones con los actores y definir cómo se registrará la información. Durante las sesiones, el analista debe escuchar más de lo que habla, pedir ejemplos concretos, verificar que ha entendido y registrar todo por escrito, ya sea en actas, grabaciones autorizadas o esquemas.

Cada sesión debe cerrar con un resumen de lo acordado y una lista de dudas pendientes. Enviar ese resumen a los participantes en las horas siguientes permite corregir malentendidos cuando todavía están frescos. Las técnicas específicas se estudian en el capítulo 5.

4.3 Análisis y elaboración

Una vez recopilada la información, se procede a analizarla. El analista clasifica los requerimientos (funcionales, no funcionales, reglas de negocio), elimina duplicados, agrupa los que se relacionan y detecta omisiones, ambigüedades y contradicciones.

En esta etapa se construyen también modelos que ayudan a comprender el problema, como diagramas de casos de uso, diagramas de flujo de datos, diagramas de actividades o modelos de clases y entidades (Pressman, 2010).

El análisis incluye una revisión de viabilidad: no todo lo que se desea puede lograrse con la tecnología, el presupuesto o el tiempo disponibles. Cuando un requerimiento resulta poco realista, debe discutirse con los actores para replantearlo o descartarlo, en lugar de aceptarlo y descubrir el problema durante el desarrollo.

4.4 Negociación y priorización

Como se indicó anteriormente, es frecuente que los actores tengan intereses contrapuestos y que los recursos no alcancen para satisfacerlos a todos. La negociación consiste en discutir los conflictos, explicar las implicaciones de cada decisión (costo, tiempo, riesgo) y llegar a acuerdos aceptables para las partes. El analista actúa aquí como facilitador, no como árbitro: su papel es poner las alternativas sobre la mesa y ayudar a que quienes tienen la autoridad decidan con información suficiente. Los criterios de priorización se estudian en el capítulo 7.

4.5 Especificación

La especificación consiste en documentar los requerimientos acordados de forma clara, ordenada y verificable, de manera que sirvan de referencia común para el cliente, los diseñadores, los programadores y los responsables de pruebas. Puede adoptar la forma de un documento formal de especificación de requerimientos de software (ERS), de un conjunto de historias de usuario con criterios de aceptación o de una combinación de ambos, según la metodología del proyecto. Este tema se desarrolla en el capítulo 6.

4.6 Validación

La validación comprueba que los requerimientos documentados describen realmente lo que los actores necesitan. Sommerville (2011) recomienda verificar, entre otros aspectos, su validez, consistencia, completitud, realismo y verificabilidad. Para ello se emplean revisiones formales con los actores, prototipos que permiten ver y probar una versión preliminar, y la elaboración anticipada de

casos de prueba: si no es posible imaginar una prueba para un requerimiento, probablemente este sea ambiguo.

4.7 Gestión de cambios y trazabilidad

Los requerimientos cambian durante el proyecto: el negocio evoluciona, aparecen normas nuevas, los usuarios comprenden mejor lo que necesitan al ver avances. Lo peligroso no es el cambio, sino el cambio sin control. La gestión de requerimientos establece un procedimiento para proponer, evaluar, aprobar e incorporar modificaciones, y mantiene una línea base, es decir, una versión acordada de los requerimientos que sirve de punto de comparación.

La trazabilidad, por su parte, permite seguir cada requerimiento desde su origen hasta el diseño, el código y las pruebas que lo verifican. Gracias a ella es posible saber, por ejemplo, qué elementos se verán afectados si se modifica un requerimiento, o si todos los requerimientos aprobados han sido probados. La Tabla 3 muestra un ejemplo sencillo de matriz de trazabilidad.

Tabla 3
Ejemplo de matriz de trazabilidad

ID	Fuente	Caso de uso	Caso de prueba
RF-01	Entrevista con la bibliotecaria	CU-01 Registrar préstamo	CP-01, CP-02
RF-03	Encuesta a estudiantes	CU-03 Buscar libro	CP-05
RNF-02	Política de seguridad de la universidad	Todos los casos de uso del módulo administrativo	CP-09

Nota. Elaboración propia.

5. Técnicas de recopilación de información

Existen numerosas técnicas para obtener requerimientos. Zowghi y Coulin (2005) presentan una revisión de las técnicas, enfoques y herramientas disponibles y concluyen que ninguna es suficiente por sí sola: cada una tiene fortalezas y limitaciones, y lo habitual es combinar varias. Goguen y Linde (1993), por su parte, destacan la importancia de comprender el contexto social en el que trabajan las

personas, no solo sus declaraciones. A continuación, se describen las técnicas de uso más extendido.

5.1 Entrevistas

La entrevista es la técnica más utilizada. Consiste en una conversación planificada entre el analista y uno o más actores con el propósito de conocer sus necesidades, sus tareas y sus problemas actuales. Sommerville (2011) distingue las entrevistas cerradas, en las que se responde a un conjunto predefinido de preguntas, y las abiertas, en las que se exploran los temas con más libertad. En la práctica se suele recurrir a un formato semiestructurado: se prepara una guía de temas y preguntas, pero se permite que la conversación derive hacia aspectos no previstos.

Para que una entrevista sea productiva conviene definir su objetivo, investigar previamente al entrevistado y su área, preparar preguntas abiertas ("¿cómo realiza actualmente este proceso?") y de seguimiento ("¿qué ocurre cuando falta ese dato?"), evitar preguntas que sugieran la respuesta y agendar sesiones de duración razonable. Sus ventajas son la riqueza de la información y la posibilidad de aclarar dudas al instante; sus limitaciones, el tiempo que consumen, el riesgo de que el entrevistado use un lenguaje técnico ajeno al analista y la dificultad de que las personas expresen conocimientos que aplican de forma automática. En el Anexo A se presenta una guía modelo.

5.2 Cuestionarios y encuestas

Los cuestionarios permiten recopilar información de muchas personas en poco tiempo y con un costo bajo. Pueden contener preguntas cerradas (opción múltiple, escalas de valoración) que se tabulan con facilidad, y preguntas abiertas que aportan matices. Son especialmente útiles cuando los usuarios son numerosos o están geográficamente dispersos, como los estudiantes de una universidad con varias sedes.

Su principal limitación es que no permiten profundizar: si una respuesta es ambigua, no hay oportunidad de aclararla en el momento. Además, la calidad de los resultados depende de la redacción de las preguntas y de la tasa de respuesta. Por

ello suelen emplearse como complemento de las entrevistas, por ejemplo, para confirmar con una muestra amplia lo que se descubrió con unos pocos usuarios clave.

5.3 Observación y etnografía

La observación consiste en presenciar a los usuarios mientras realizan su trabajo en su entorno real, sin interferir. Su mayor valor es que revela lo que las personas hacen en la práctica, que con frecuencia difiere de lo que dicen que hacen. La etnografía es una forma sistemática de observación, tomada de las ciencias sociales, en la que el analista se integra durante un período al entorno de trabajo para comprender sus rutinas, su cultura y sus interacciones. Sommerville (2011) señala que es particularmente eficaz para descubrir requerimientos implícitos que surgen de la forma en que las personas colaboran y que ellas mismas no logran describir.

Entre sus desventajas están el tiempo que requiere, la posibilidad de que las personas cambien su comportamiento al saberse observadas y su menor utilidad para requerimientos de un sistema completamente nuevo, cuyos procesos aún no existen.

5.4 Talleres de trabajo (JAD) y lluvia de ideas

Los talleres reúnen en una misma sesión a representantes de los distintos actores para definir requerimientos de manera colaborativa. La técnica de Diseño Conjunto de Aplicaciones (JAD, por sus siglas en inglés) fue desarrollada por IBM a finales de la década de 1970 y se basa en sesiones estructuradas, con un facilitador, objetivos claros y participantes con capacidad de decisión. Su principal ventaja es que los conflictos entre actores se discuten y se resuelven en el momento, lo que acorta el ciclo de acuerdos.

La lluvia de ideas es una técnica que suele emplearse dentro de los talleres: se invita a los participantes a proponer libremente ideas, sin criticarlas, y luego se agrupan y se seleccionan. Los talleres exigen una buena coordinación de agendas

y un facilitador experimentado, pues corren el riesgo de que dominen las opiniones de los participantes más influyentes o más habladores.

5.5 Análisis de documentos y de sistemas existentes

Revisar los documentos de la organización (manuales de procedimientos, formularios, reportes, contratos, normas internas) y estudiar los sistemas que se utilizan actualmente proporciona información valiosa antes de reunirse con los actores. Permite conocer el vocabulario, los datos que se manejan y las reglas de negocio, y llegar a las entrevistas con preguntas más precisas. También ayuda a identificar deficiencias del sistema actual que deben corregirse en el nuevo. Su limitación es que la documentación puede estar desactualizada o no reflejar la práctica real.

5.6 Prototipos

Un prototipo es una versión preliminar y simplificada del sistema, o de una parte de él, que se construye para explorar requerimientos. Puede ser de baja fidelidad (bocetos en papel o esquemas de pantallas) o de alta fidelidad (maquetas interactivas que simulan la interfaz). Sommerville (2011) señala que los usuarios comprenden mejor lo que quieren, y descubren lo que no quieren, cuando pueden ver y manipular algo tangible en lugar de leer una descripción.

Los prototipos pueden ser desechables, cuando se construyen solo para aprender y luego se descartan, o evolutivos, cuando se refinan hasta convertirse en el producto final. Su riesgo es que el cliente los confunda con el sistema terminado y espere plazos irreales, o que el equipo pierda tiempo puliendo detalles visuales en lugar de aclarar lo esencial.

5.7 Escenarios, casos de uso e historias de usuario

Estas técnicas describen la interacción entre el usuario y el sistema mediante situaciones concretas. Un escenario narra una secuencia de acciones, por ejemplo, la de un estudiante que busca un libro y lo reserva. Los casos de uso, propuestos por Ivar Jacobson a finales de la década de 1980, formalizan estos escenarios identificando a los actores, el objetivo que persiguen y los pasos para lograrlo

(Cockburn, 2001). Las historias de usuario, muy usadas en las metodologías ágiles, expresan una necesidad con una frase breve desde la perspectiva del usuario (Cohn, 2004). Ambas se estudian con más detalle en el capítulo 6, ya que sirven tanto para obtener como para documentar requerimientos.

5.8 Criterios para seleccionar las técnicas

La elección depende de factores como el número y la ubicación de los actores, el tiempo y el presupuesto disponibles, la madurez del dominio, la novedad del sistema y el tipo de información que se busca. Por ejemplo, para un sistema que reemplaza a otro existente, el análisis de documentos y la observación aportan mucho; para un producto completamente nuevo, los prototipos y los talleres resultan más útiles. La Tabla 4 compara las técnicas descritas.

Tabla 4
Comparación de las técnicas de recopilación de información

Técnica	Ventajas	Limitaciones	Cuándo conviene usarla
Entrevistas	Información detallada; permiten aclarar dudas	Consumen tiempo; dependen de la habilidad del entrevistador	Al inicio y con actores clave
Cuestionarios	Alcanzan a muchas personas a bajo costo	Poca profundidad; riesgo de baja respuesta	Muchos usuarios o dispersos
Observación	Revela la práctica real del trabajo	Lenta; puede alterar el comportamiento	Procesos existentes complejos
Talleres (JAD)	Resuelve conflictos en el momento	Difícil coordinar agendas; requiere facilitador	Actores con intereses distintos
Análisis de documentos	Rápido; aporta vocabulario y reglas	Puede estar desactualizado	Preparación previa a entrevistas
Prototipos	Los usuarios reaccionan ante algo concreto	Pueden crear expectativas irreales	Interfaces y requerimientos poco claros
Casos de uso e historias	Describen el sistema desde el usuario	Requieren disciplina para redactarlos	Documentar y validar funciones

Nota. Elaboración propia con base en Sommerville (2011) y Zowghi y Coulin (2005).

6. Documentación y especificación de requerimientos

6.1 El documento de especificación de requerimientos de software (ERS)

El documento de especificación de requerimientos de software, conocido como ERS (o SRS, por sus siglas en inglés), es el acuerdo escrito entre el cliente y el equipo de desarrollo sobre lo que el sistema debe hacer. Sirve de referencia para el diseño, para la planificación, para las pruebas de aceptación y, en muchos casos, como base contractual. La norma IEEE 830-1998 fue durante años el modelo más difundido para estructurarlo y actualmente ha sido reemplazada por la norma ISO/IEC/IEEE 29148 (2018), que integra y amplía sus recomendaciones. La estructura clásica propuesta por la IEEE (1998) es la siguiente:

1. **Introducción:** propósito, alcance, definiciones y abreviaturas, referencias y visión general del documento.
2. **Descripción general:** perspectiva del producto, funciones principales, características de los usuarios, restricciones, suposiciones y dependencias.
3. **Requerimientos específicos:** interfaces externas, funciones, requerimientos de rendimiento, restricciones de diseño y atributos de calidad del sistema.
4. **Apéndices e índice:** información de apoyo que facilita la consulta del documento.

La norma también describe las características de una buena especificación: debe ser correcta, no ambigua, completa, consistente, clasificada por importancia, verificable, modificable y trazable (IEEE, 1998). Un ERS excesivamente extenso o que nadie lee no cumple su propósito; por eso su nivel de detalle debe adaptarse al tamaño y al riesgo del proyecto.

6.2 Características de un buen requerimiento

Además de la calidad del documento en su conjunto, cada requerimiento individual debe cumplir ciertos criterios. La norma ISO/IEC/IEEE 29148 (2018) señala, entre otros, que un requerimiento debe ser necesario, apropiado, no ambiguo, completo,

singular, factible, verificable, correcto y conforme con los estándares aplicables. En términos prácticos, esto significa que:

- **Es necesario:** responde a una necesidad real de un actor; si se elimina, el sistema pierde valor.
- **No es ambiguo:** admite una sola interpretación; evita palabras como “rápido”, “fácil” o “adecuado” sin medida.
- **Es completo:** contiene toda la información necesaria para entenderlo, sin depender de suposiciones.
- **Es singular:** expresa una sola idea; si contiene “y”, es probable que sean dos requerimientos.
- **Es factible:** puede lograrse con los recursos, la tecnología y el tiempo disponibles.
- **Es verificable:** existe una forma objetiva de comprobar que se cumple, mediante una prueba, una inspección o una medición.

Un ejemplo permite apreciar la diferencia. El enunciado “el sistema debe ser rápido” es ambiguo y no verificable, pues cada persona entiende algo distinto por rápido. En cambio, “el sistema mostrará los resultados de una búsqueda en menos de 3 segundos con hasta 200 usuarios simultáneos” es preciso y puede medirse. De igual manera, “el sistema registrará préstamos y enviará recordatorios” mezcla dos ideas y debe dividirse en dos requerimientos independientes.

6.3 Formas de redactar los requerimientos

Lenguaje natural estructurado. Es la forma más común. Se redactan frases cortas con una estructura constante, como “El sistema deberá…”, y se distinguen los niveles de obligatoriedad mediante verbos: “deberá” para lo obligatorio, “debería” para lo deseable y “podrá” para lo opcional. Se acompaña de un identificador único (RF-01, RNF-02) que facilita la trazabilidad.

Casos de uso. Describen una interacción completa entre un actor y el sistema para alcanzar un objetivo. Cockburn (2001) propone una plantilla con el nombre del caso,

el actor principal, las precondiciones, el flujo principal de eventos, los flujos alternativos y las postcondiciones. La Tabla 5 presenta un ejemplo abreviado.

Tabla 5
Ejemplo de caso de uso: Registrar préstamo de libro

Elemento	Descripción
Identificador	CU-01
Actor principal	Bibliotecario
Objetivo	Registrar la salida de un libro para un estudiante.
Precondiciones	El bibliotecario ha iniciado sesión; el estudiante está registrado y no tiene sanciones.
Flujo principal	1. El bibliotecario ingresa la cédula del estudiante. 2. El sistema muestra sus datos y sus préstamos vigentes. 3. El bibliotecario escanea el código del libro. 4. El sistema verifica que el libro esté disponible. 5. El sistema registra el préstamo y calcula la fecha de devolución. 6. El sistema muestra un comprobante.
Flujos alternativos	4a. Si el libro no está disponible, el sistema informa y ofrece registrar una reserva. 2a. Si el estudiante ya tiene tres libros prestados, el sistema no permite continuar.
Postcondiciones	El préstamo queda registrado y el libro cambia a estado “prestado”.

Nota. Elaboración propia con base en el formato de Cockburn (2001).

Historias de usuario. Muy usadas en las metodologías ágiles, expresan un requerimiento desde el punto de vista del usuario con una fórmula breve: “Como [rol], quiero [funcionalidad] para [beneficio]”. Cohn (2004) sugiere que una buena historia cumple criterios conocidos por la sigla INVEST: independiente, negociable, valiosa, estimable, pequeña y comprobable. Cada historia se acompaña de criterios de aceptación, que pueden redactarse con el formato “Dado – Cuando – Entonces”. Por ejemplo: “Como estudiante, quiero buscar libros por título, autor o materia para saber si están disponibles sin ir a la biblioteca”. Su criterio de aceptación podría ser: “Dado que estoy en la página de búsqueda, cuando escribo un título existente, entonces el sistema muestra el libro y su disponibilidad”.

6.4 Ejemplo de especificación

Para ilustrar lo anterior, la Tabla 6 presenta un fragmento de especificación para el sistema de préstamo de libros de una biblioteca universitaria. Cada requerimiento tiene un identificador, una redacción singular y verificable, un tipo y una prioridad.

Tabla 6
Fragmento de especificación de requerimientos: sistema de préstamos de biblioteca

ID	Descripción	Tipo	Prioridad
RF-01	El sistema permitirá al bibliotecario registrar el préstamo de un libro a un estudiante.	Funcional	Debe
RF-02	El sistema permitirá al estudiante buscar libros por título, autor o materia.	Funcional	Debe
RF-03	El sistema enviará un correo electrónico al estudiante dos días antes de la fecha de devolución.	Funcional	Debería
RF-04	El sistema calculará la multa por cada día de retraso en la devolución.	Funcional	Podría
RNF-01	Las búsquedas mostrarán resultados en menos de 3 segundos con hasta 200 usuarios simultáneos.	No funcional	Debería
RNF-02	El acceso al módulo de administración requerirá autenticación con usuario y contraseña.	No funcional	Debe
RNF-03	La interfaz podrá utilizarse desde computadoras y teléfonos móviles.	No funcional	Debería

Nota. Elaboración propia (ejemplo ilustrativo).

7. Priorización y validación de requerimientos

7.1 ¿Por qué priorizar?

En casi todos los proyectos la lista de requerimientos supera lo que puede construirse con el tiempo y el presupuesto disponibles. Si todo se declara “urgente”, el equipo carece de criterio para decidir qué hacer primero o qué sacrificar cuando surgen problemas. Priorizar permite entregar primero lo más valioso, reducir riesgos y negociar con fundamento cuando se solicitan cambios. Las decisiones deben tomarse con los actores y no unilateralmente por el equipo técnico, ya que el valor de un requerimiento lo determina quien obtiene el beneficio.

7.2 La técnica MoSCoW

MoSCoW es una de las técnicas de priorización más difundidas. Fue propuesta por Dai Clegg y se popularizó con el método de desarrollo de sistemas dinámicos DSDM (Clegg & Barker, 1994). Su nombre proviene de las iniciales de cuatro categorías; las letras “o” se agregan solo para facilitar la pronunciación. La Tabla 7 resume su significado.

Tabla 7
Categorías de la técnica MoSCoW

Categoría	Significado	Ejemplo en el sistema de biblioteca
Must have (Debe tener)	Indispensable; sin él el sistema no cumple su propósito.	Registrar préstamos y devoluciones.
Should have (Debería tener)	Importante, pero existe una alternativa temporal si no está.	Recordatorios por correo electrónico.
Could have (Podría tener)	Deseable; se incluye si hay tiempo y presupuesto.	Cálculo automático de multas.
Won’t have (No tendrá por ahora)	Acordado que queda fuera de esta entrega.	Aplicación móvil nativa.

Nota. Elaboración propia con base en Clegg y Barker (1994).

Para que la técnica funcione es recomendable que la categoría “debe” no absorba toda la capacidad del equipo; de lo contrario, no queda margen para imprevistos. Otras técnicas de uso frecuente son la votación con puntos (cada actor distribuye una cantidad limitada de puntos entre los requerimientos que considera más importantes) y la matriz de valor y esfuerzo, que compara el beneficio esperado de cada requerimiento con el costo de implementarlo.

7.3 Validación de los requerimientos

La validación busca responder a una pregunta esencial: ¿estamos definiendo el sistema correcto? No debe confundirse con la verificación, que comprueba que el producto construido cumple con lo especificado. Sommerville (2011) recomienda revisar en cada requerimiento su validez (¿responde a una necesidad real?), su consistencia (¿se contradice con otro?), su completitud, su realismo (¿es posible construirlo con los recursos disponibles?) y su verificabilidad.

Las técnicas de validación más habituales son:

- **Revisiones e inspecciones:** reuniones en las que actores, analistas y desarrolladores leen el documento buscando errores, omisiones y ambigüedades, a menudo con listas de verificación.
- **Prototipos:** permiten que los usuarios comprueben si el sistema propuesto realmente se ajusta a lo que necesitan.
- **Generación de casos de prueba:** redactar las pruebas de aceptación desde la etapa de requerimientos revela defectos de redacción; un requerimiento que no puede probarse debe reescribirse.
- **Recorridos con los usuarios:** simulaciones paso a paso de los casos de uso o escenarios más importantes.

Cuando finaliza la validación, los actores aprueban formalmente la versión de los requerimientos que servirá como línea base del proyecto. A partir de ese momento, cualquier modificación se tramita mediante el procedimiento de gestión de cambios descrito en el capítulo 4.

8. El levantamiento en los enfoques tradicionales y ágiles

8.1 Enfoque tradicional

En los modelos secuenciales, como el modelo en cascada, el levantamiento se concentra al inicio del proyecto. Se busca definir todos los requerimientos antes de pasar al diseño y producir un ERS completo que, una vez aprobado, se toma como referencia contractual. Este enfoque funciona bien cuando los requerimientos son estables y bien comprendidos, o cuando el contexto exige documentación exhaustiva, como en sistemas sujetos a regulaciones estrictas. Su debilidad es que los errores y omisiones pueden pasar inadvertidos durante meses, hasta que el usuario ve el sistema en funcionamiento.

8.2 Enfoque ágil

Las metodologías ágiles parten de la premisa de que los requerimientos cambian y de que es imposible conocerlos por completo al inicio. El Manifiesto por el Desarrollo

Ágil de Software (Beck et al., 2001) prioriza la colaboración con el cliente por encima de la negociación contractual y la respuesta al cambio por encima del seguimiento de un plan. En este enfoque, los requerimientos se registran como historias de usuario en una lista priorizada llamada *backlog* del producto, que se revisa y refina continuamente. En cada iteración el equipo selecciona las historias más prioritarias, las construye y las muestra a los usuarios, quienes aportan retroalimentación que alimenta nuevas historias.

Esto no significa que el levantamiento desaparezca: se distribuye a lo largo del proyecto en lugar de concentrarse al inicio. Tampoco significa ausencia de documentación, sino documentación suficiente y de bajo costo: historias, criterios de aceptación, prototipos y modelos sencillos.

8.3 Comparación

La Tabla 8 resume las diferencias principales en la manera de abordar el levantamiento de requerimientos en ambos enfoques.

Tabla 8
Levantamiento de requerimientos: enfoque tradicional frente a enfoque ágil

Aspecto	Enfoque tradicional	Enfoque ágil
Momento del levantamiento	Concentrado al inicio del proyecto	Continuo, en cada iteración
Documentación	ERS extenso y formal	Historias de usuario, *backlog* y criterios de aceptación
Manejo de cambios	Control formal de cambios	Cambios bienvenidos y priorizados en el *backlog*
Participación del cliente	Principalmente en hitos de revisión y aprobación	Colaboración constante con el equipo
Detección de malentendidos	Tardía, al probar o entregar	Temprana, al revisar cada incremento
Contexto ideal	Requerimientos estables, contratos cerrados, regulación estricta	Requerimientos inciertos o cambiantes, necesidad de entregas rápidas

Nota. Elaboración propia con base en Sommerville (2011), Cohn (2004) y Beck et al. (2001).

En la práctica muchas organizaciones adoptan enfoques híbridos: realizan un levantamiento inicial para comprender el contexto y el alcance general, y luego

refinan los detalles de manera iterativa. Lo esencial, en cualquier caso, es que exista un entendimiento compartido de lo que se va a construir y un mecanismo ordenado para gestionar los cambios.

9. Problemas frecuentes, buenas prácticas y herramientas

9.1 Problemas frecuentes

Pressman (2010) agrupa las dificultades del levantamiento en problemas de alcance (límites del sistema mal definidos o detalles técnicos innecesarios), de comprensión (los actores no están seguros de lo que necesitan o no dominan el vocabulario del otro) y de volatilidad (los requerimientos cambian con el tiempo). En la práctica estas dificultades se manifiestan de formas concretas:

- **Ambigüedad:** requerimientos redactados con términos vagos que cada persona interpreta a su manera.
- **Ampliación descontrolada del alcance:** se agregan funciones nuevas sin evaluar su impacto en costo y tiempo.
- **Requerimientos tácitos:** conocimientos que los usuarios aplican de forma automática y que no mencionan porque los consideran obvios.
- **Poca disponibilidad de los actores:** los usuarios clave no tienen tiempo para participar y el equipo decide por suposición.
- **Conflictos entre actores:** necesidades contrapuestas que no se negocian abiertamente y aparecen más tarde.
- **Barreras de comunicación:** uso de jerga técnica o del negocio que no comparten todos los participantes.
- **Exceso de requerimientos:** solicitudes de funciones que aportan poco valor pero encarecen el proyecto.
- **Falta de trazabilidad:** no se sabe de dónde surgió un requerimiento ni qué elementos del sistema dependen de él.

9.2 Buenas prácticas

Sommerville y Sawyer (1997) y Wiegers y Beatty (2013) recopilan prácticas que mejoran la calidad del proceso. Entre las más relevantes están:

- Involucrar a los usuarios desde el inicio y mantener su participación hasta la validación.
- Elaborar un glosario de términos compartido por todo el equipo y los actores.
- Combinar varias técnicas de obtención y contrastar la información de fuentes distintas.
- Registrar las reuniones y enviar resúmenes a los participantes para su confirmación.
- Redactar cada requerimiento con un identificador único, con criterios verificables y con una prioridad asignada.
- Usar modelos y prototipos como apoyo, ya que muchas personas comprenden mejor lo visual que lo escrito.
- Establecer una línea base y un procedimiento formal para evaluar y aprobar los cambios.
- Mantener la trazabilidad de los requerimientos hacia el diseño, el código y las pruebas.
- Revisar los requerimientos con inspecciones antes de iniciar el diseño.

9.3 Herramientas de apoyo

Existen herramientas que facilitan el trabajo con los requerimientos. Para el registro y seguimiento de historias y tareas se utilizan Jira, Azure DevOps o Trello; para la documentación colaborativa, Confluence, Notion o los editores de documentos en línea; para el diseño de prototipos, Figma o Balsamiq; para elaborar diagramas, draw.io, Lucidchart o Visual Paradigm. En proyectos grandes o sujetos a normativas estrictas se emplean herramientas especializadas de gestión de requerimientos, como IBM DOORS o Jama Connect, que ofrecen control de versiones y trazabilidad automatizada. La herramienta no reemplaza al proceso: una hoja de cálculo bien organizada es preferible a una aplicación sofisticada mal utilizada.

9.4 Consideraciones éticas y de inclusión

El levantamiento también implica responsabilidades éticas. El analista maneja información sensible sobre las personas y sobre la organización, por lo que debe respetar la confidencialidad y el consentimiento al grabar entrevistas o publicar resultados. Debe, además, procurar que se escuchen todas las voces relevantes y no solo la de quien tiene más autoridad, y considerar desde el inicio requerimientos de accesibilidad para personas con discapacidades y de protección de los datos personales. Un sistema que excluye a parte de sus usuarios por no haberlos consultado es, en el fondo, el resultado de un levantamiento incompleto.

CONCLUSIONES Y RECOMENDACIONES

Luis Madrid

Conclusiones

El levantamiento de requerimientos es la etapa que define el rumbo de un proyecto: de su calidad dependen el diseño, las pruebas y la satisfacción de los usuarios.

Un requerimiento es una necesidad comprendida y expresada con claridad; por eso, levantar requerimientos implica descubrir, interpretar y acordar, y no solo recolectar solicitudes.

Recomendaciones

Dedicar tiempo suficiente al levantamiento antes de iniciar el diseño, definiendo desde el principio el alcance, los actores y el calendario de reuniones.

Detectar los errores de requerimientos lo más temprano posible, ya que corregirlos tarde es mucho más costoso.

José Sousa

Conclusiones

Distinguir entre requerimientos de negocio, de usuario y de sistema, y entre funcionales y no funcionales, permite organizar la información y evitar que aspectos como el rendimiento o la seguridad se descubran demasiado tarde.

Identificar a todos los actores y comprender su nivel de interés e influencia es indispensable para obtener información completa y anticipar conflictos que deberán negociarse.

Recomendaciones

Analizar los requerimientos no funcionales desde el inicio del proyecto y no dejarlos para el final.

Elaborar al comenzar una lista de actores y una matriz de interés e influencia, para saber a quién consultar y con qué frecuencia.

Roderick Justiniani

Conclusiones

El proceso de levantamiento es iterativo: sus etapas se repiten y se solapan, y volver a etapas anteriores cuando surgen cambios es normal, no un fallo.

No existe una técnica que sirva para todos los casos; combinar entrevistas, cuestionarios, observación, talleres, análisis de documentos y prototipos produce resultados más confiables.

Recomendaciones

Elegir las técnicas según el contexto del proyecto (número de usuarios, tiempo, presupuesto y novedad del sistema) y combinar más de una.

Preparar guías de preguntas y enviar a los participantes un resumen de cada reunión para corregir malentendidos a tiempo.

Luis Montalvo

Conclusiones

Documentar los requerimientos con criterios de calidad (claros, verificables, singulares, priorizados y trazables) reduce las ambigüedades y facilita la validación con los usuarios.

Tanto los enfoques tradicionales como los ágiles necesitan levantar requerimientos; cambian el momento y el nivel de formalidad, pero en ambos es indispensable gestionar los cambios de forma ordenada.

Recomendaciones

Redactar cada requerimiento con un identificador único, una sola idea y criterios medibles, evitando términos vagos como “rápido” o “fácil”, y priorizarlos con técnicas como MoSCoW.

Establecer desde el inicio una línea base, un procedimiento de control de cambios y una matriz de trazabilidad, incluso en proyectos pequeños.

INFOGRAFÍA


LEVANTAMIENTO DE REQUERIMIENTOS
Cómo descubrir lo que el software realmente debe hacer
Ingeniería de Software Aplicada I · Universidad Tecnológica de Panamá
1 ¿Qué es?
Es la actividad de descubrir, comprender, documentar y acordar con los actores involucrados las necesidades que un sistema de software debe satisfacer, antes de diseñarlo y construirlo.
2 Tipos de requerimientos
Funcionales
Qué hace el sistema
"El sistema permitirá registrar un préstamo de libro."
No funcionales
Cómo se comporta
"La consulta responderá en menos de 3 segundos con 200 usuarios."
De dominio y restricciones
Límites y normas
"Los datos personales se protegerán según la normativa aplicable."
3 El proceso en 6 pasos
1 Identificar
2 Obtener
3 Analizar
4 Negociar
5 Especificar
6 Validar
Todo el proceso se acompaña de gestión de cambios y trazabilidad
4 Técnicas para obtener requerimientos
Entrevistas
Cuestionarios
Observación
Talleres (JAD)
Prototipos
Casos de uso e historias de usuario
5 Un buen requerimiento es...
Necesario
Claro, sin ambigüedad
Completo
Verificable
Consistente
Factible
6 Priorización MoSCoW
Must
Debe estar
Should
Debería estar
Could
Podría estar
Won't
Ahora no
Elaboración propia con base en Sommerville (2011), Pressman (2010), Wiegers y Beatty (2013) e ISO/IEC/IEEE 29148 (2018).

BIBLIOGRAFÍA

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifiesto por el desarrollo ágil de software*. https://agilemanifesto.org/iso/es/manifesto.html

Bourque, P., & Fairley, R. E. (Eds.). (2014). *Guide to the software engineering body of knowledge (SWEBOK)* (Version 3.0). IEEE Computer Society. https://www.swebok.org

Brooks, F. P. (1987). No silver bullet: Essence and accidents of software engineering. *Computer, 20*(4), 10–19. https://doi.org/10.1109/MC.1987.1663532

Clegg, D., & Barker, R. (1994). *Case method fast-track: A RAD approach*. Addison-Wesley. https://dl.acm.org/doi/10.5555/561543

Cockburn, A. (2001). *Writing effective use cases*. Addison-Wesley. https://www.informit.com/store/writing-effective-use-cases-9780201702255

Cohn, M. (2004). *User stories applied: For agile software development*. Addison-Wesley. https://books.google.com/books/about/User_Stories_Applied.html?id=SvIwuX4SVigC

Goguen, J. A., & Linde, C. (1993). Techniques for requirements elicitation. En *Proceedings of the IEEE International Symposium on Requirements Engineering* (pp. 152–164). IEEE. https://ieeexplore.ieee.org/document/324822

IEEE. (1998). *IEEE recommended practice for software requirements specifications* (IEEE Std 830-1998). IEEE. https://ieeexplore.ieee.org/document/720574

ISO/IEC/IEEE. (2018). *Systems and software engineering — Life cycle processes — Requirements engineering* (ISO/IEC/IEEE 29148:2018). ISO. https://www.iso.org/standard/72089.html

Kendall, K. E., & Kendall, J. E. (2011). *Análisis y diseño de sistemas* (8.ª ed.). Pearson Educación. https://www.vitalsource.com/products/analisis-y-diseno-de-sistemas-kenneth-e-kendall-v9786073205771

Kotonya, G., & Sommerville, I. (1998). *Requirements engineering: Processes and techniques*. John Wiley & Sons. https://www.wiley.com/en-us/Requirements+Engineering:+Processes+and+Techniques-p-9780471972082

Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico* (7.ª ed.). McGraw-Hill. https://latam.casadellibro.com/libro-ingenieria-del-software-7-ed/9786071503145/1775061

Project Management Institute. (2014). *Pulse of the profession: Requirements management — A core competency for project and program success*. https://www.pmi.org/learning/thought-leadership/pulse/core-competency-project-program-success

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación. https://www.pearsoneducacion.net/mexico/tienda-online/ingenieria-software-sommerville-9ed-libro

Sommerville, I., & Sawyer, P. (1997). *Requirements engineering: A good practice guide*. John Wiley & Sons. https://www.wiley.com/en-us/Requirements+Engineering:+A+Good+Practice+Guide-p-9780471974444

Wiegers, K., & Beatty, J. (2013). *Software requirements* (3rd ed.). Microsoft Press. https://www.microsoftpressstore.com/store/software-requirements-9780735679665

Zowghi, D., & Coulin, C. (2005). Requirements elicitation: A survey of techniques, approaches, and tools. En A. Aurum & C. Wohlin (Eds.), *Engineering and*

managing software requirements (pp. 19–46). Springer. https://doi.org/10.1007/3-540-28244-0_2

ANEXOS

Anexo A. Guía modelo para una entrevista de levantamiento

La siguiente guía puede adaptarse según el proyecto y el entrevistado. Las preguntas son abiertas para invitar a la explicación, y se deben complementar con preguntas de seguimiento.

Tabla 9
Guía modelo de entrevista

Bloque	Preguntas sugeridas
1. Apertura	Presentación del analista y del objetivo de la reunión. Confirmar el tiempo disponible y solicitar autorización para tomar notas o grabar.
2. Contexto del entrevistado	¿Cuál es su cargo y cuáles son sus responsabilidades principales? ¿Con qué frecuencia interactúa con este proceso?
3. Proceso actual	¿Cómo realiza actualmente esta tarea, paso a paso? ¿Qué información utiliza y de dónde proviene? ¿Qué documentos o sistemas emplea?
4. Problemas y oportunidades	¿Qué partes del proceso le resultan más lentas o complicadas? ¿Qué errores ocurren con más frecuencia? ¿Qué haría diferente si pudiera?
5. Expectativas del nuevo sistema	¿Qué debería poder hacer el sistema? ¿Qué información necesita ver y con qué frecuencia? ¿Qué reportes necesita?
6. Restricciones y reglas	¿Existen normas, políticas o leyes que deban cumplirse? ¿Quiénes tienen permiso para realizar cada acción?
7. Prioridades	Si solo pudiera contar con tres funciones al inicio, ¿cuáles elegiría? ¿Qué sería inaceptable que faltara?
8. Cierre	¿Hay algo que no le he preguntado y que considere importante? ¿Con quién más debería hablar? Resumen de lo acordado y próximos pasos.

Nota. Elaboración propia.

Anexo B. Plantilla de ficha de requerimiento

Esta plantilla permite registrar cada requerimiento con la información necesaria para analizarlo, priorizarlo y rastrearlo.

Tabla 10
Plantilla de ficha de requerimiento

Campo	Contenido
Identificador	Código único (por ejemplo, RF-01 o RNF-01).
Nombre	Título breve que resuma el requerimiento.
Descripción	Enunciado claro, singular y verificable (“El sistema deberá…”).
Tipo	Funcional, no funcional, regla de negocio, restricción o interfaz.
Fuente	Actor, documento o norma de donde proviene.
Prioridad	Debe, debería, podría o no tendrá por ahora (MoSCoW).
Criterio de aceptación	Condición objetiva que permite comprobar que se cumple.
Dependencias	Otros requerimientos con los que se relaciona.
Estado	Propuesto, aprobado, implementado, verificado o rechazado.
Versión y fecha	Número de versión y fecha del último cambio.
Responsable	Persona encargada de mantener el requerimiento.

Nota. Elaboración propia.

Anexo C. Glosario de términos

Tabla 11
Glosario de términos del levantamiento de requerimientos

Término	Definición
Actor (*stakeholder*)	Persona, grupo u organización que afecta o es afectada por el sistema.
Analista de requerimientos	Profesional que obtiene, analiza, documenta y valida los requerimientos.
Backlog	Lista priorizada de historias de usuario u otros elementos de trabajo de un producto.
Caso de uso	Descripción de la interacción entre un actor y el sistema para lograr un objetivo.
Criterio de aceptación	Condición que debe cumplirse para considerar aceptado un requerimiento o una historia.
ERS (o SRS)	Documento de especificación de requerimientos de software.
Historia de usuario	Descripción breve de una necesidad desde la perspectiva del usuario.
Levantamiento de requerimientos	Conjunto de actividades para descubrir y comprender las necesidades de los actores.
Línea base	Versión aprobada de los requerimientos que sirve de referencia para controlar los cambios.
Prototipo	Versión preliminar del sistema utilizada para explorar y validar requerimientos.
Requerimiento funcional	Servicio o comportamiento que el sistema debe ofrecer.
Requerimiento no funcional	Propiedad o restricción sobre cómo opera el sistema (rendimiento, seguridad, usabilidad, etc.).
Trazabilidad	Capacidad de seguir un requerimiento desde su origen hasta su implementación y prueba.
Validación	Comprobación de que los requerimientos describen lo que los actores realmente necesitan.

Nota. Elaboración propia con base en las fuentes consultadas.